1. Importing Libraries

```
import tensorflow as tf
import re
from pickle import load,dump
import os
from os import listdir
from numpy import array,argmax
from IPython.display import Image,display


from tensorflow.keras.applications.vgg16 import VGG16,preprocess_input
from tensorflow.keras.preprocessing.image import load_img,img_to_array
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical

from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical

from tensorflow.keras.layers import Input,Dense,LSTM,Embedding,Dropout, add
from tensorflow.keras.callbacks import ModelCheckpoint

from nltk.translate.bleu_score import corpus_bleu
```

2. Downloading Datasets

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("youssefaboelnasr/flickr8k-text")
```

```
Downloading to /root/.cache/kagglehub/datasets/youssefaboelnasr/flickr8k-text/1.a
100%|██████████| 2.25M/2.25M [00:01<00:00, 1.37MB/s]Extracting files...
```

```
all_captions = os.path.join(path, "Flickr8k.lemma.token.txt")
test_captions = os.path.join(path, "Flickr_8k.testImages.txt")
train_captions = os.path.join(path, "Flickr_8k.trainImages.txt")
```

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("adityajn105/flickr8k")

print("Path to dataset files:", path)
```

```
Using Colab cache for faster access to the 'flickr8k' dataset.
Path to dataset files: /kaggle/input/flickr8k
```

```
import shutil
import os
```

```
source_path_images = path # Assuming 'path' still holds the location of the flick
destination_dir_images = '/content/sample_data/2/'

# Ensure the parent destination directory exists
os.makedirs(destination_dir_images, exist_ok=True)

# Construct the full destination path, keeping the dataset folder name
destination_path_images = os.path.join(destination_dir_images, os.path.basename(s

# If the destination directory already exists, remove it to prevent errors from s
if os.path.exists(destination_path_images):
    shutil.rmtree(destination_path_images)

# Copy the directory from the read-only source to the destination
shutil.copytree(source_path_images, destination_path_images)

print(f"Image dataset copied from '{source_path_images}' to '{destination_path_ir

# Update the 'path' variable to track the new location
path = destination_path_images
```

```
Image dataset copied from '/kaggle/input/flickr8k' to '/content/sample_data/2/flic
```

```
all_images= "/content/sample_data/2/flickr8k/Images"
```

```
listdir(all_images)
```

```
'233290142_5b1aedf0a7.jpg'
```

```
 223299142_521aedf9e7.jpg',
 '1295669416_21cabf594d.jpg',
 '3678100844_e3a9802471.jpg',
 '2444134813_20895c640c.jpg',
 '1564614124_0ee6799935.jpg',
 '1057210460_09c6f4c6c1.jpg',
 '320093980_5388cb3733.jpg',
 '3694555931_7807db2fb4.jpg',
 '3364160101_c5e6c52b25.jpg',
 '2187720319_112d00f07d.jpg',
 '1801063894_60bce29e19.jpg',
 '3638783120_f600ceb19d.jpg',
 '871290666_4877e128c0.jpg',
 '3172384527_b107385a20.jpg',
 '272156850_c4445a53f4.jpg',
 '3562302012_0cbcd01ff9.jpg',
 '3066429707_842e50b8f7.jpg',
 '2344412916_9a5a9b1c82.jpg',
 '2616284322_b13e7c344e.jpg',
 '2070798293_6b9405e04d.jpg',
 '1452361926_6d8c535e32.jpg',
 '3486135177_772628d034.jpg',
 '3639105305_bd9cb2d1db.jpg',
 '524036004_6747cf909b.jpg',
 '1814391289_83a1eb71d3.jpg',
   ]
```

3. Preprocessing Data

```
file1=open(all_captions,"r")
data1=file1.read()
file1.close()
```

```
data1
```

```
'1305564994_00513f9a5b.jpg#0\tA man in street racer armor be examine the tire of
another racer \'s motorbike .\n1305564994_00513f9a5b.jpg#1\tTwo racer drive a whi
te bike down a road .\n1305564994_00513f9a5b.jpg#2\tTwo motorist be ride along on
their vehicle that be oddly design and color .\n1305564994_00513f9a5b.jpg#3\tTwo
person be in a small race car drive by a green hill .\n1305564994_00513f9a5b.jpg#
4\tTwo person in race uniform in a street car .\n1351764581_4d4fb1b40f.jpg#0\tA f
irefighter extinguish a fire under the hood of a car .\n1351764581_4d4fb1b40f.jpg
#1\ta fireman spray water into the hood of small white car on a jack\n1351764581_
4d4fb1b40f.jpg#2\tA fireman spray inside the open hood of small white car , on a
jack .\n1351764581_4d4fb1b40f.jpg#3\tA fireman use a firehose on a car engine tha
t be up on a carjack .\n1351764581_4d4fb1b40f.jpg#4\tFirefighter use water to ext
inguish a car that be on fire .\n1358089136_976e3d2e30.jpg#0\tA boy sand surf dow
n a hill\n1358089136_976e3d2...'
```

```
descr = dict()
for line in data1.split("\n"):
  token = line.split("\t")
  if len(line) < 2:
    continue
  image_id, image_desc = token[0], token[1]
  image_id = image_id.split(".")[0]
  image_desc = "<start> "+"".join(image_desc)+ "<end>"

  if image_id not in descr:
    descr[image_id] = list()
  descr[image_id].append(image_desc)
```

```
descr['1000268201_693b08cb0e']
```

```
['<start> A child in a pink dress be climb up a set of stair in an entry way .
<end>',
 '<start> A girl go into a wooden building .<end>',
 '<start> A little girl climb into a wooden playhouse .<end>',
 '<start> A little girl climb the stair to her playhouse .<end>',
 '<start> A little girl in a pink dress go into a wooden cabin .<end>']
```

```
for key,des in descr.items():
  for i in range(len(des)):
    d=des[i].lower()
    d = re.sub(r"\w*[^a-z\s<>]+\w*", "", d)
    des[i] = d
    descr[key] = des
```

```
vocab = set()
for k,v in descr.items():
  [vocab.update(l.split()) for l in v]
print(f"the orginal size of the vocabulary is {len(vocab)}")
```

```
the orginal size of the vocabulary is 7079
```

```
vocab
```

```
'boy'
```

```
 now',
 'backwards<end>',
 'broadway',
 'streched',
 'trash',
 'shrubbery',
 'railgrind',
 'nearly',
 'garment',
 'cupcake',
 'shelton',
 'rotary',
 'universal',
 'playroom',
 'dolly',
 'shop',
 'everybody',
 'barack',
 'scale',
 'barn',
 'dandelion',
 'amoung',
 ...}
```

```
def load_text(descr,filename):
  length = 0
  des = dict()
  with open(filename,"r") as file:
    keys = file.read()
    for k in keys.split("\n"):
      if len(k)<2:
        continue
      image_id = k.split(".")[0]
      des[image_id]=descr.get(image_id) # Corrected bug: using image_id instead
      length = length +1
  print(f"Number of image : {length}")
  return des
```

```
train_descr  = load_text(descr,train_captions)
test_descr  = load_text(descr,test_captions)
```

```
Number of image : 6000
Number of image : 1000
```

```
train_descr["3675825945_96b2916959"]
```

```
['<start> a man in skateboard in an empty swim pool <end>',
 '<start> a man ride a skateboard up the side of an empty pool <end>',
 '<start> a skateboarder dress in white be drop into a blue bowl <end>',
 '<start> a skateboarder hang off the edge of a bowl at a skate park <end>',
 '<start> a skateboarder in an empty pool <end>']
```

IMAGE DATA PREPROCESSING

```
all_images
```

```
'/content/sample_data/2/flickr8k/Images'
```

```
listdir(all_images)
```

```
'3396817186_b299ee0531.jpg',
'3134644844_493eec6cdc.jpg',
'1220401002_3f44b1f3f7.jpg',
'2501742763_b2cb322087.jpg',
'2521062020_f8b983e4b2.jpg',
'447800028_0242008fa3.jpg',
'1294578091_2ad02fea91.jpg',
'3212456649_40a3052682.jpg',
'2831846986_5534425cfa.jpg',
'3636247381_65ccf8f106.jpg',
'3648988742_888a16f600.jpg',
'3457364788_3514a52091.jpg',
'2321466753_5606a10721.jpg',
'2541701582_0a651c380f.jpg',
'3604384157_99241be16e.jpg',
'308307853_5a51fbdecc.jpg',
'386655611_1329495f97.jpg',
'766061382_6c7ff514c4.jpg',
'2765029348_667111fc30.jpg',
'3674168459_6245f4f658.jpg',
'1131340021_83f46b150a.jpg',
'1324816249_86600a6759.jpg',
'1473250020_dc829a090f.jpg',
'3616771728_2c16bf8d85.jpg',
'2268601066_b018b124fd.jpg',
'3203878596_cbb307ce3b.jpg',
'3420064875_0349a75d69.jpg',
'2899276965_a20b839cfd.jpg',
'2665904080_8a3b9639d5.jpg',
'3568197730_a071d7595b.jpg',
'2502782508_2c8211cd6b.jpg',
'3198231851_6b1727482b.jpg',
'3134387321_3a253224c1.jpg',
'3283897411_af9d0b497d.jpg',
'93922153_8d831f7f01.jpg',
'3094317837_b31cbf969e.jpg',
'1778020185_1d44c04dae.jpg',
'352981175_16ff5c07e4.jpg',
'3522025527_c10e6ebd26.jpg',
'2170187328_65c2f11891.jpg',
'2826769554_85c90864c9.jpg',
'1417637704_572b4d6557.jpg',
'3550459890_161f436c8d.jpg',
'633456174_b768c1d6cd.jpg',
'1980315248_82dbc34676.jpg',
'2308108566_2cba6bca53.jpg',
'2848977044_446a31d86e.jpg',
'3500136982_bf7a85531e.jpg',
'340667199_ecae5f6029.jpg',
'2373234213_4ebe9c4ee5.jpg',
'3322200641_c2e51ff37b.jpg',
'2051194177_fbeee211e3.jpg',
'1396064003_3fd949c9dd.jpg',
'640053014_549d2f23d2.jpg',
'2553619107_d382a820f9.jpg',
'2508249781_36e9282423.jpg',
'2097489021_ca1b9f5c3b.jpg',
'3404870997_7b0cd755de.jpg',
'543363241_74d8246fab.jpg',
```

```
model = VGG16()
model.layers.pop()
model = Model(inputs=model.inputs, outputs=model.layers[-1].output)
features = dict()
for name in listdir(all_images):
```

```
  filename = os.path.join(all_images, name) # Use os.path.join for robust path c
  if not os.path.isfile(filename): # Check if it's a file
      print(f"Skipping {filename} as it is not a file.")
      continue
  try:
    image = load_img(filename, target_size=(224, 224))
    image = img_to_array(image)
    image=image.reshape((1,image.shape[0],image.shape[1],image.shape[2] ))
    image = preprocess_input(image)
    feature = model.predict(image,verbose=0)
    image_id = name.split(".")[0]
    features[image_id]=feature
  except (UnidentifiedImageError, FileNotFoundError):
    print(f"Could not identify or find image file: {filename}. Skipping.")
    continue

print(f"The number of extracted features: {len(features)}")
dump(features, open("features.dump","wb"))
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-application
553467096/553467096 ━━━━━━━━━━━━━━━━━━━━ 33s 0us/step
The number of extracted features: 8091
```

```
def load_img_feat(fn_features,fn_data):
  length = 0
  feat =  dict()
  file1= open(fn_data,"r")
  file2= open(fn_features,"rb")
  f = load(file2)
  keys = file1.read()
  for k in keys.split("\n"):
    if len(k)<2:
      continue
    image_id = k.split(".")[0] # Corrected line: use 'k' instead of 'image_id'
    feat[image_id] = f[image_id]
    length +=1
  file1.close()
  file2.close()
  print(f"No of Instances {length} ")
  return feat
```

```
train_features=load_img_feat("/content/features.dump",train_captions)
test_features=load_img_feat("/content/features.dump",test_captions)
```

```
No of Instances 6000
No of Instances 1000
```

```
test_features["2943334864_6bab479a3e.jpg"]
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/tmp/ipykernel_7788/3925892545.py in <cell line: 0>()
----> 1 test_features["2943334864_6bab479a3e.jpg"]

KeyError: '2943334864_6bab479a3e.jpg'
```

Next steps: Explain error

Text Tokenization:

```
def tokenizer(descr):
  d = []
  for key in descr.keys():
    for i in descr[key]:
      d.append(i)
  tokenize = Tokenizer()
  tokenize.fit_on_texts(d)
  return tokenize
```

```
tokenize = tokenizer(train_descr)
vocab_size = len(tokenize.word_index)+1
```

```
print(vocab_size)
```

```
5463
```

```
d = []
for key in train_descr.keys():
    for i in train_descr[key]:
        d.append(i)

maxlen = max(len(i.split()) for i in d)
print(f"max length if caption in training data is {maxlen}")
```

```
max length if caption in training data is 36
```

```
def create_sequences(tokenizer,max_length,desc_list,photo,vocab_size):
  x1,x2,y = list(),list(),list()
  for desc in desc_list:
    seq = tokenizer.texts_to_sequences([desc])[0]
    for i in range(1,len(seq)):
      in_seq,out_seq = seq[:i],seq[i]
      in_seq = pad_sequences([in_seq],maxlen=max_length)[0]

      out_seq = to_categorical([out_seq],num_classes = vocab_size)[0]
      x1.append(photo) # Corrected: using 'photo' argument instead of global 'im
      x2.append(in_seq)
      y.append(out_seq)
  return array(x1),array(x2),array(y)
```

```
def data_generator(descriptions, images, tokenizer, max_length, vocab_size):
    while 1:
        for key, desc_list in descriptions.items():
            image_features = images[key][0]
            in_img, in_seq, out_word = create_sequences(tokenizer, max_length, d

            # Ensure that actual sequences were generated before yielding
            if len(in_img) > 0:
                yield (in_img, in_seq), out_word
```

```
def define_model(vocab_size, max_length):
    inputs1 = Input(shape=(1000,))
    fe1 = Dropout(0.5)(inputs1)
    fe2 = Dense(256, activation="relu")(fe1)

    # Sequence model
    inputs2 = Input(shape=(max_length,))
    se1 = Embedding(vocab_size, 256)(inputs2)
    se2 = Dropout(0.5)(se1)
    se3 = LSTM(256)(se2)

    decoder1 = add([fe2, se3])
    decoder2 = Dense(256, activation="relu")(decoder1)
    outputs = Dense(vocab_size, activation="softmax")(decoder2)

    model = Model(inputs = [inputs1, inputs2], outputs=outputs)

    model.compile(loss="categorical_crossentropy", optimizer="adam")
    print(model.summary())
    return model
```

```
#Train the model
model = define_model(vocab_size, maxlen)
epochs = 10
steps = len(train_descr)
for i in range(epochs):
    generator = data_generator(train_descr, train_features, tokenize, maxlen, vo

    model.fit(generator, epochs=1, steps_per_epoch=steps, verbose=1)
    model.save("model_" + str(i) + ".h5")
```

```
Model: "functional_15"
```

Layer (type)	Output Shape	Param #	Connected to
input_layer_30 (InputLayer)	(None, 36)	0	-
input_layer_29 (InputLayer)	(None, 1000)	0	-
embedding_14 (Embedding)	(None, 36, 256)	1,398,528	input_layer_30[0…
dropout_28 (Dropout)	(None, 1000)	0	input_layer_29[0…
dropout_29 (Dropout)	(None, 36, 256)	0	embedding_14[0][…
dense_42 (Dense)	(None, 256)	256,256	dropout_28[0][0]
lstm_14 (LSTM)	(None, 256)	525,312	dropout_29[0][0]
add_14 (Add)	(None, 256)	0	dense_42[0][0], lstm_14[0][0]
dense_43 (Dense)	(None, 256)	65,792	add_14[0][0]
dense_44 (Dense)	(None, 5463)	1,403,991	dense_43[0][0]

```
 Total params: 3,649,879 (13.92 MB)
 Trainable params: 3,649,879 (13.92 MB)
 Non-trainable params: 0 (0.00 B)
None
6000/6000 ━━━━━━━━━━━━━━━━━━━━ 78s 12ms/step - loss: 3.8932
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `kera
6000/6000 ━━━━━━━━━━━━━━━━━━━━ 74s 12ms/step - loss: 3.2688
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `kera
6000/6000 ━━━━━━━━━━━━━━━━━━━━ 75s 12ms/step - loss: 3.0542
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `kera
6000/6000 ━━━━━━━━━━━━━━━━━━━━ 80s 13ms/step - loss: 2.9205
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `kera
6000/6000 ━━━━━━━━━━━━━━━━━━━━ 74s 12ms/step - loss: 2.8268
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `kera
6000/6000 ━━━━━━━━━━━━━━━━━━━━ 75s 12ms/step - loss: 2.7588
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `kera
6000/6000 ━━━━━━━━━━━━━━━━━━━━ 75s 12ms/step - loss: 2.7021
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `kera
6000/6000 ━━━━━━━━━━━━━━━━━━━━ 75s 12ms/step - loss: 2.6558
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `kera
6000/6000 ━━━━━━━━━━━━━━━━━━━━ 80s 13ms/step - loss: 2.6175
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `kera
6000/6000 ━━━━━━━━━━━━━━━━━━━━ 81s 13ms/step - loss: 2.5872
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `kera
```

```
def word_for_id(integer,tokenizer):
  for word,index in tokenizer.word_index.items():
    if index == integer:
      return word
  return None
```

```
#generate description for an image
def generate_desc(model, tokenizer, photo, max_length):
    in_text = "<start>"
    for i in range(max_length):
        sequence = tokenizer.texts_to_sequences([in_text])[0]
        sequence = pad_sequences([sequence], maxlen=max_length)
        y_hat = model.predict([photo, sequence], verbose=0)
        y_hat = argmax(y_hat)
        word = word_for_id(y_hat, tokenizer)
        if word is None:
            break
        in_text += " " + word
        if word == "end":
            break
    return in_text
```

```
def evaluate_model(model, descriptions, photos, tokenizer, max_length):
    actual, predicted = list(), list()
    for key, desc_list in descriptions.items():
        yhat = generate_desc(model, tokenizer, photos[key], max_length)
        references = [d.split() for d in desc_list]
        actual.append(references)
        predicted.append(yhat.split())
    print("BLEU-1: %f" % corpus_bleu(actual, predicted, weights=(1.0,0,0,0)))
```

```
evaluate_model(model,test_descr,test_features,tokenize,maxlen)
```

```
BLEU-1: 0.529852
```

```
display(Image("/content/sample_data/2/flickr8k/Images/3385593926_d3e9c21170.jpg"
```

```
yhat=generate_desc(model,tokenize,test_features["3385593926_d3e9c21170"],maxlen)
print(yhat)
```

```
<start> a brown dog be run through the grass end
```

```
    maxlength=maxlen
```

```
    dump(tokenizer, open("tokenizer.pkl","wb"))
    model = VGG16()
    # pop the last layer to take the output from last second layer
```